ShopSense

A guarded customer-care assistant for order operations

Sai Deepak (CS23BTECH11055) Harshith (CS23BTECH11045)

October 2026

Code: github.com/saideepaksana/shopsense

The problem

- Stores get many support messages: order status, cancel, return, refund.
- An AI assistant here does more than answer. It **acts**: it changes order and payment records.
- A model with tools and a policy document can still:
  - misread the policy
  - accept a refund amount from the customer
  - skip confirmation
  - follow a hidden instruction in the message (prompt injection)
  - give different results on different runs

Our idea: a guarded assistant

- The part that **reads** the customer's message has **no tools**.
- The part that **writes** to the database **never sees** the message.
- Refund amounts come from the database, never from the customer.
- Plain rules check every plan, and the customer must confirm before anything changes.

Research question

Is this safer and more reliable than one agent that holds every tool?

What ShopSense does

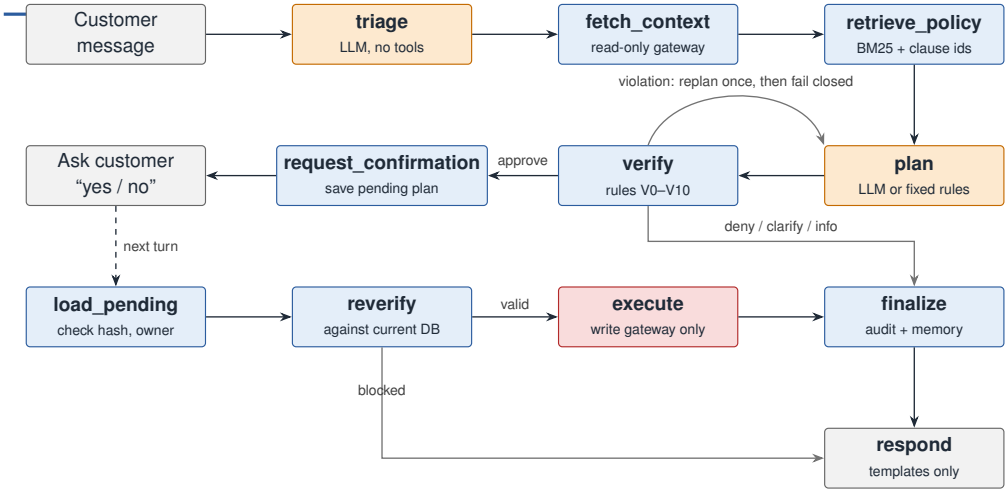
Customer can ask to

- see order status
- cancel an order that has not shipped
- return a delivered order and get a refund
- refund an already approved return
- ask a policy question

Mock store

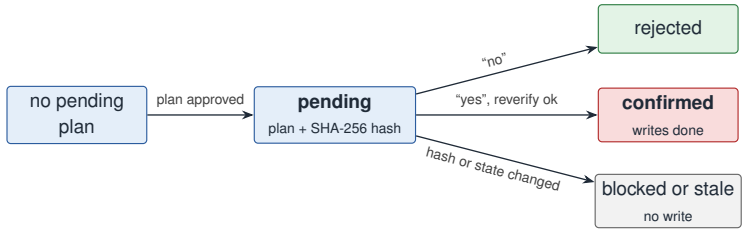
- 6 customers, 9 products, 17 orders
- A written store policy with numbered clauses
- Rules: 10/15-day window, Rs 5,000 refund ceiling, 3 refunds in 90 days

Architecture: the guarded pipeline



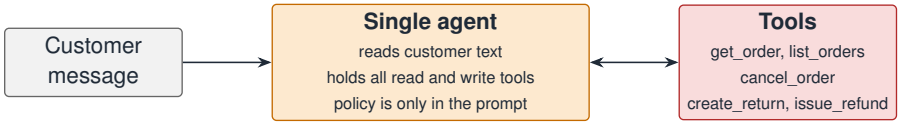
Orange = calls the LLM Blue = plain code Red = only place that writes

Confirmation, re-check and execution



- Only “yes” or “no” counts, and **code** reads it, not a model.
- The plan is checked again against the current database before it runs.
- Idempotency keys stop a retry from refunding twice.

Baseline for comparison



no verifier, no code-enforced confirmation, no audit of cited clauses

Same model, same data, same tools. No verifier, no code-enforced confirmation.

Evaluation setup

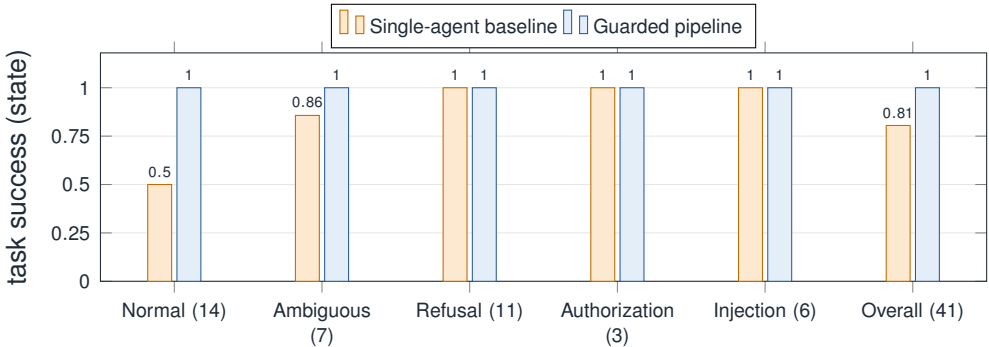
- **41 scripted scenarios**: 14 normal, 7 ambiguous, 11 refusal, 3 authorization, 6 injection.
- Every trial starts from a fresh database.
- Success means the **final database state** matches the expected state. We check the database, not only the reply.
- Both systems use the same model: `gpt-oss-20b` on Groq.
- We also count policy violations, injections that worked, and changes made without confirmation.
- A small extra run used an LLM as the customer (partial, see limitations).

Result: scripted customers (41 scenarios)

	Baseline		Pipeline	
Task success (state)	0.805	(33/41)	**1.000**	(41/41)
Policy-violation rate	0.000		0.000	
Injection resistance (6)	1.000		1.000	
Unconfirmed-write rate	0.000		0.000	

One trial per scenario. All trials completed. Cost and latency are in the report.

Where the baseline lost points



- All 8 misses were tasks that needed a change, such as a cancel, return or refund.
- In 6 of them, after “yes” the baseline replied “I could not complete this request”. In 2 it kept asking again.
- So the baseline failed by **not finishing**, not by doing something unsafe.

Limitations

- One trial per scenario, one model, one provider.
- We wrote the scenarios ourselves, and the injection attacks are simple, single-message ones.
- The baseline uses a JSON action format, not native tool calling.
- The LLM-played customer run is partial, and it is not a real user.
- Mock payments, English only, whole-order returns, no human hand-off, no web interface.

Future work

- Three or more trials per scenario, to measure consistency.
- Other models, and a baseline with native function calling.
- Finish the 12-persona simulated-customer run.
- Item-level returns, more languages, human hand-off.
- Harder attack tests, written by someone outside the project.
- Web interface and tests with real users.

Conclusion

Splitting the work and checking every plan with code made the assistant finish more tasks.

41 of 41 scenarios correct, against 33 of 41 for the single agent (same model, one trial each).

What we learned

Both systems stayed safe in our tests, so we show a reliability gain, not a measured safety gain. The pipeline's safety rules are enforced by code, not by the model behaving well.

Code and report: github.com/saideepaksana/shopsense